

Q1. In a dummy-headed doubly circular linked list, which statement is TRUE?
Answer: C) Tail's next points to dummy head
Explanation: Since the list is circular, the last node links back to the dummy head.

Q2. In a dummy-headed doubly circular list, how do you detect the end of traversal?
Answer: C) current == head
Explanation: Traversal ends when we return to the dummy head node.

Q3. In a doubly linked list, the worst-case time complexity to insert a node at the beginning (given head reference) is O (___1___).
Explanation: Only a few pointer updates are required.

Q4. The worst-case time complexity of deleting the last node in a doubly linear linked list (when tail is not given) is O (___n___).
Explanation: We must traverse the list to find the last node.

Q5. In a dummy-headed doubly circular linked list, the time complexity of deleting the last node (given only head) is O (___1___).
Explanation: The last node is directly accessible via head . prev.

Q6. In a dummy-headed doubly linear linked list, the worst-case time complexity to insert at the end (without tail pointer) is O (___n___).
Explanation: Traversal is needed to reach the last node.

Java	Python
<pre>public static void printForward(Node head) { if (head.next == null) { System.out.println("Empty"); return; } Node temp = head.next; // Skip dummy head while (temp != null) { System.out.print(temp.elem + " "); temp = temp.next; } System.out.println(); }</pre>	<pre>def printForward(head): if head.next is None: print("Empty") return temp = head.next # Skip dummy head while temp is not None: print(temp.elem, end=" ") temp = temp.next print()</pre>

<pre> public static void printBackward(Node head) { if (head.next == null) { System.out.println("Empty"); return; } // Move to the last node Node temp = head.next; while (temp.next != null) { temp = temp.next; } // Traverse backward until reaching the dummy head while (temp != head) { System.out.print(temp.elem + " "); temp = temp.prev; } System.out.println(); } </pre>	<pre> def printBackward(head): if head.next is None: print("Empty") return # Move to the last node temp = head.next while temp.next is not None: temp = temp.next # Traverse backward until reaching the dummy head while temp != head: print(temp.elem, end=" ") temp = temp.prev print() </pre>
--	--

Java	Python
<pre> public static Node oddReverse(Node head) { Node newHead = null; Node current = head; while (current != null) { Node nextNode = current.next; if (current.elem % 2 != 0) { current.next = newHead; newHead = current; } current = nextNode; } return newHead; } </pre>	<pre> def oddReverse(head): newHead = None current = head while current is not None: nextNode = current.next if current.elem % 2 != 0: current.next = newHead newHead = current current = nextNode return newHead </pre>
<pre> public static Node evenReverse(Node head) { Node newHead = null; Node current = head; while (current != null) { Node nextNode = current.next; if (current.elem % 2 == 0) { current.next = newHead; newHead = current; } current = nextNode; } return newHead; } </pre>	<pre> def evenReverse(head): newHead = None current = head while current is not None: nextNode = current.next if current.elem % 2 == 0: current.next = newHead newHead = current current = nextNode return newHead </pre>

<pre> public static Node lessThanXReverse(Node head, int X) { Node newHead = null; Node current = head; while (current != null) { Node nextNode = current.next; if (current.elem < X) { current.next = newHead; newHead = current; } current = nextNode; } return newHead; } </pre>	<pre> def lessThanXReverse(head, X): newHead = None current = head while current is not None: nextNode = current.next if current.elem < X: current.next = newHead newHead = current current = nextNode return newHead </pre>
<pre> public static Node greaterThanXReverse(Node head, int X) { Node newHead = null; Node current = head; while (current != null) { Node nextNode = current.next; if (current.elem > X) { current.next = newHead; newHead = current; } current = nextNode; } return newHead; } </pre>	<pre> def greaterThanXReverse(head, X): newHead = None current = head while current is not None: nextNode = current.next if current.elem > X: current.next = newHead newHead = current current = nextNode return newHead </pre>

Solution by calling reverseInPlace() directly
<pre> public static Node reverseInPlace(Node head) { Node prev = null; // For changing the direction of the link Node current = head; // To traverse the linked list while (current != null) { Node nextNode = current.next; // Store the next node current.next = prev; // Reverse the link prev = current; // Move prev forward current = nextNode; // Move current forward } return prev; // New head of the reversed linked list } </pre>

```

/*
 * Removes all odd nodes and reverses
 * the remaining even nodes.
 */
public static Node evenReverse(Node head) {

    Node pred = null; // Previous retained node
    Node rem = head; // Node being examined or removed

    while (rem != null) {

        // Store the successor before modifying any link
        Node succ = rem.next;

        // Remove rem if it contains an odd value
        if (rem.elem % 2 != 0) {

            // Special case: rem is the head
            if (pred == null) {
                head = succ;
            }

            // General case: rem is after pred
            else {
                pred.next = succ;
            }

            // Disconnect the removed node
            rem.next = null;
        }

        // Retain rem if it contains an even value
        else {
            pred = rem;
        }

        // Move rem forward
        rem = succ;
    }

    // Reverse the remaining even nodes
    return reverseInPlace(head);
}

```

```

/*
 * Removes all even nodes and reverses
 * the remaining odd nodes.
 */
public static Node oddReverse(Node head) {

    Node pred = null; // Previous retained node
    Node rem = head; // Node being examined or removed

    while (rem != null) {

        // Store the successor before modifying any link
        Node succ = rem.next;

        // Remove rem if it contains an even value
        if (rem.elem % 2 == 0) {

```

```

    // Special case: rem is the head
    if (pred == null) {
        head = succ;
    }

    // General case: rem is after pred
    else {
        pred.next = succ;
    }

    // Disconnect the removed node
    rem.next = null;
}

// Retain rem if it contains an odd value
else {
    pred = rem;
}

// Move rem forward
rem = succ;
}

// Reverse the remaining odd nodes
return reverseInPlace(head);
}

```

```

/*
 * Removes all nodes containing values greater than
 * or equal to X and reverses the remaining nodes.
 */
public static Node lessThanXReverse(Node head, int X) {

    Node pred = null; // Previous retained node
    Node rem = head; // Node being examined or removed

    while (rem != null) {

        // Store the successor before modifying any link
        Node succ = rem.next;

        // Remove rem if its value is not less than X
        if (rem.elem >= X) {

            // Special case: rem is the head
            if (pred == null) {
                head = succ;
            }

            // General case: rem is after pred
            else {
                pred.next = succ;
            }

            // Disconnect the removed node
            rem.next = null;
        }

        // Retain rem if its value is less than X
        else {
            pred = rem;
        }
    }
}

```

```

    }

    // Move rem forward
    rem = succ;
}

// Reverse the remaining nodes
return reverseInPlace(head);
}

```

```

/*
 * Removes all nodes containing values less than
 * or equal to X and reverses the remaining nodes.
 */
public static Node greaterThanXReverse(Node head, int X) {

    Node pred = null; // Previous retained node
    Node rem = head; // Node being examined or removed

    while (rem != null) {

        // Store the successor before modifying any link
        Node succ = rem.next;

        // Remove rem if its value is not greater than X
        if (rem.elem <= X) {

            // Special case: rem is the head
            if (pred == null) {
                head = succ;
            }

            // General case: rem is after pred
            else {
                pred.next = succ;
            }

            // Disconnect the removed node
            rem.next = null;
        }

        // Retain rem if its value is greater than X
        else {
            pred = rem;
        }

        // Move rem forward
        rem = succ;
    }

    // Reverse the remaining nodes
    return reverseInPlace(head);
}

```

Time complexity: $O(n^2)$ ---Solution is correct-but does not maintain restriction--marks will be deducted

```
public static Node copyEvenReverse(Node head) {
    Node tail = head;

    // Remove all odd nodes
    while (tail != null) {

        if (tail.elem % 2 != 0) {

            Node succ = tail.next;

            // Special case: tail is the head node
            if (tail == head) {
                head = succ;
            }

            // General case: tail has a predecessor
            else {
                Node pred = getPred(head, tail);
                pred.next = succ;
            }

            // Disconnect the removed node
            tail.next = null;

            // Move tail to the successor
            tail = succ;
        }

        else {
            // Keep the even node and move forward
            tail = tail.next;
        }
    }

    // Reverse the remaining even nodes
    return reverseInPlace(head);
}

public static Node getPred(Node head, Node target) {

    // The head node has no predecessor
    if (head == null || head == target) {
        return null;
    }

    Node current = head;

    while (current != null) {

        // current is the predecessor of target
        if (current.next == target) {
            return current;
        }

        current = current.next;
    }

    // Target was not found
    return null;
}
```